

WEB APPLICATION PENETRATION TESTING REPORT

DVWA Security Assessment

Project: CloudExify Cybersecurity Project 3

Assessment Type: Advanced Web Application Penetration Testing

Target: Damn Vulnerable Web Application (DVWA)

Environment: Local / Authorized Testing Environment

Testing Tools: Burp Suite, Web Browser, DVWA

Tester: Aliza Javed

Date: 31August 2026

❖ Executive Summary

This report presents the results of an authorized security assessment performed against Damn Vulnerable Web Application (DVWA), an intentionally vulnerable web application designed for security testing and educational purposes.

The objective of this assessment was to identify and demonstrate common web application vulnerabilities, analyze their security impact, collect supporting evidence, and provide appropriate remediation recommendations.

The assessment focused primarily on vulnerabilities associated with the OWASP Top 10, including SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), authentication-related behavior, session management, and other relevant web application security weaknesses.

Testing was performed exclusively against the local DVWA environment. No third-party systems, external websites, or unauthorized infrastructure were targeted.

The assessment successfully demonstrated SQL Injection, Reflected XSS, Stored XSS, and CSRF vulnerabilities in the tested DVWA configuration. Authentication-related testing was also performed; however, a successful authentication bypass was not established during the assessment. Session behavior was inspected as part of the testing process.

The identified vulnerabilities demonstrate the potential for unauthorized data access, client-side script execution, and unauthorized actions performed within an authenticated user's session.

❖ **Assessment Scope**

In-Scope Target

The following application was included in the assessment:

Damn Vulnerable Web Application (DVWA)

The application was hosted within a local authorized testing environment.

Testing Scope

The assessment covered:

- SQL Injection
- Reflected Cross-Site Scripting
- Stored Cross-Site Scripting
- Cross-Site Request Forgery
- Authentication behavior
- Session management
- File upload functionality where applicable
- HTTP request interception and manipulation using Burp Suite
- Vulnerability evidence collection
- Security remediation recommendations

Out-of-Scope

The following were outside the scope of this assessment:

- Third-party systems
- Public websites
- Real user accounts
- Production systems
- Denial-of-Service testing
- Destructive exploitation
- Real-world credential attacks

❖ Testing Environment

The assessment was conducted in a controlled local environment.

Environment Components

Component	Description
Target Application	Damn Vulnerable Web Application (DVWA)
Web Server	Apache
Database	MySQL
Application Runtime	PHP
Testing Proxy	Burp Suite
Browser	Google Chrome
Network	Local/private testing environment
DVWA Security Level	Low

The target was intentionally configured as a vulnerable training application. Therefore, vulnerabilities demonstrated during testing should not be interpreted as evidence that the underlying software or configuration is suitable for production use.

❖ Methodology

The assessment followed a structured web application penetration-testing methodology.

Phase 1 — Reconnaissance and Environment Validation

The local DVWA installation was validated and accessed through the browser. The application was configured at the Low security level to enable controlled demonstration of intentionally vulnerable functionality.

Phase 2 — Request Interception

Burp Suite was used to intercept browser-generated HTTP requests.

The Proxy and HTTP history functionality were used to inspect:

- HTTP methods
- Request parameters
- Cookies
- Session identifiers
- Form submissions
- Authentication requests

Phase 3 — Vulnerability Testing

Individual DVWA modules were tested according to their intended functionality.

Testing included:

- SQL Injection
 - Reflected XSS
 - Stored XSS
 - CSRF
 - Authentication behavior
 - Session behavior
-

❖ Finding 01 — SQL Injection

Description

SQL Injection is a vulnerability that occurs when untrusted user input is incorporated into SQL queries without appropriate parameterization or other protective controls.

During testing, the DVWA SQL Injection functionality accepted manipulated input that altered the application's database query behavior.

A SQL Injection payload was successfully used to demonstrate that the application did not safely handle the supplied input.

Test Location

DVWA → SQL Injection

Testing Performed

A normal request was first tested using a valid numeric identifier.

The input was then modified using a SQL Injection expression designed to alter the logical condition of the application's SQL query.

The application returned records that demonstrated the effect of the injected SQL expression.

Evidence

Figure 1 — Normal SQL Injection Request

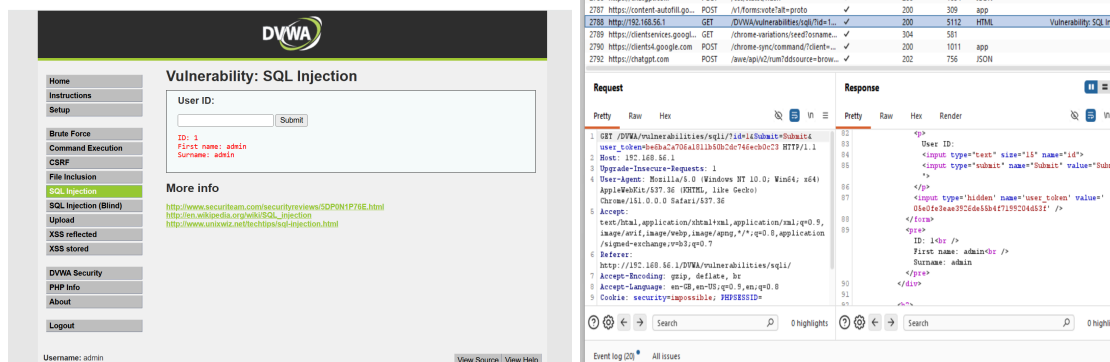
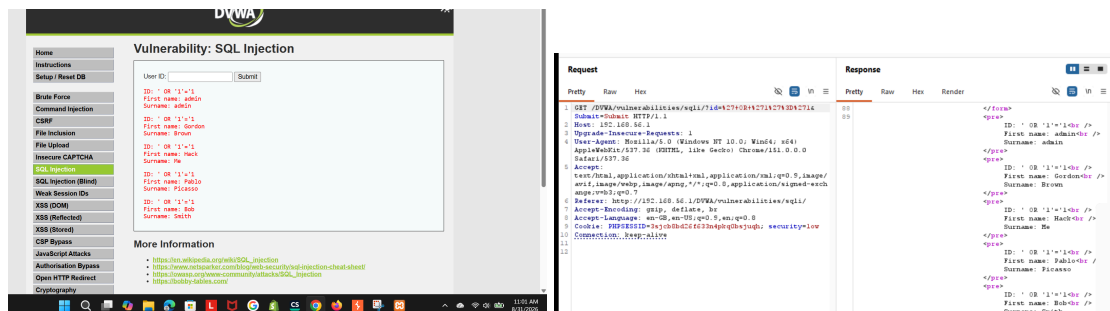


Figure 2 — SQL Injection Successful Demonstration



The evidence demonstrates that manipulated input affected the application's database query processing.

❖ Finding 02 — Reflected Cross-Site Scripting

Description

Reflected Cross-Site Scripting (XSS) occurs when user-controlled input is immediately reflected in an application's response without appropriate output encoding.

During testing of DVWA's Reflected XSS functionality, JavaScript supplied through the input field was executed by the browser.

Test Location

DVWA → XSS (Reflected)

Testing Performed

A normal text value was submitted first to establish expected application behavior.

A JavaScript test payload was subsequently supplied as the input.

The browser executed the supplied JavaScript and displayed a JavaScript alert.

Evidence

Figure 3 — Reflected XSS Normal Input

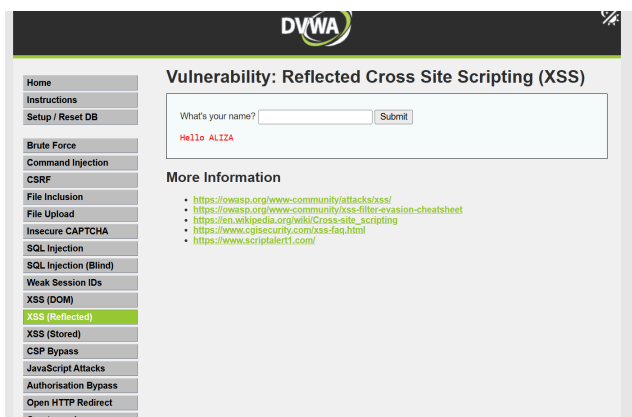
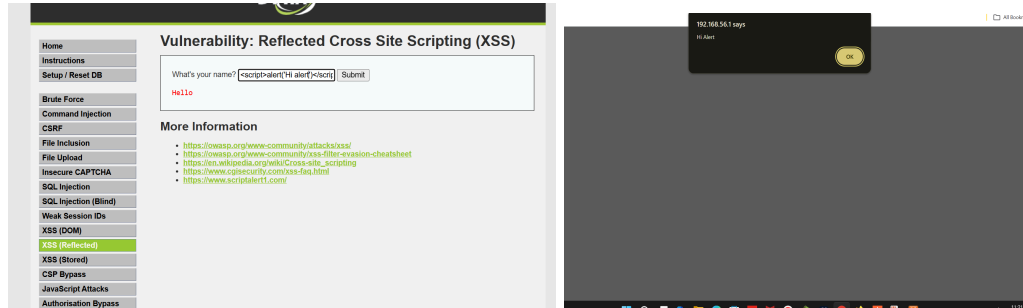


Figure 4 — Reflected XSS Successful Execution



❖ Finding 03 — Stored Cross-Site Scripting

Description

Stored XSS occurs when malicious user-controlled content is stored by an application and subsequently rendered to users without appropriate output encoding.

Unlike reflected XSS, the malicious content persists within the application's data or storage mechanism and may execute whenever the affected content is viewed.

Test Location

DVWA → XSS (Stored)

Testing Performed

A normal message was submitted to confirm that the application stored and displayed user-generated content.

A JavaScript test payload was then submitted.

The application stored the supplied content and the browser subsequently executed the JavaScript when the content was displayed.

Evidence

Figure 5 — Stored XSS Normal Message

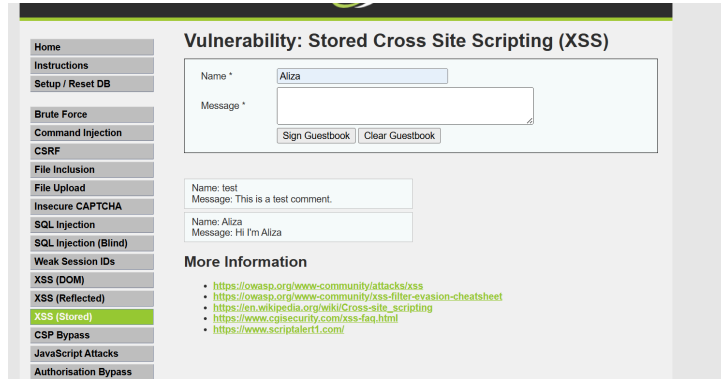
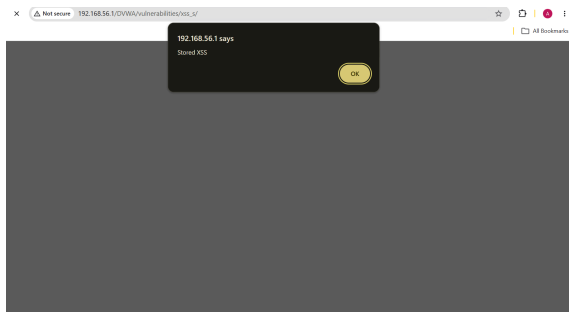


Figure 6 — Stored XSS Successful Execution



❖ Finding 04 — Cross-Site Request Forgery

Description

Cross-Site Request Forgery (CSRF) occurs when an application accepts a state-changing request from an authenticated user without adequately verifying that the request was intentionally initiated by that user.

The DVWA CSRF functionality was tested by inspecting the password-change request through Burp Suite.

Test Location

DVWA → CSRF

Testing Performed

The password-change functionality generated a GET request containing password parameters:

password_new

password_conf

Change

The intercepted request did not contain a dedicated CSRF token.

The request was subsequently replayed using Burp Suite Repeater and the application successfully processed the password-change action.

Evidence

Figure 7 — CSRF Request Intercepted in Burp Suite

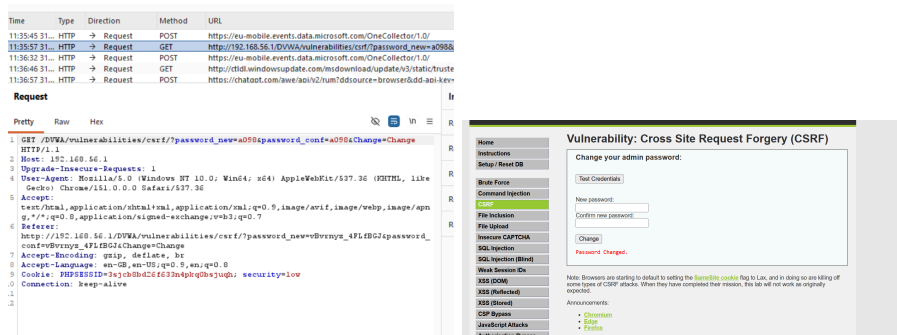
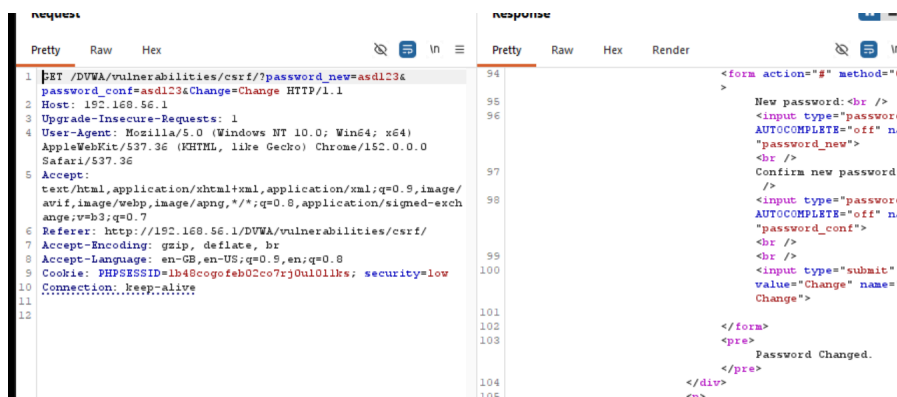


Figure 8 — Successful Password Change Response



❖ Authentication Testing

Objective

Authentication testing was performed to determine whether the DVWA authentication mechanism could be bypassed using manipulated credentials or HTTP requests.

Testing Performed

A login request was intercepted through Burp Suite.

The request contained:

- Username
- Password
- Login parameter
- `user_token`
- Session cookie

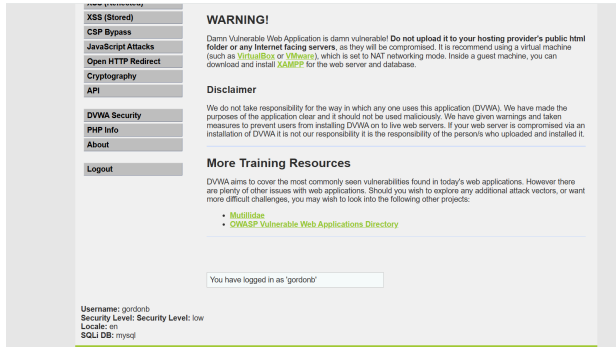
An invalid username/password combination was tested.

Result

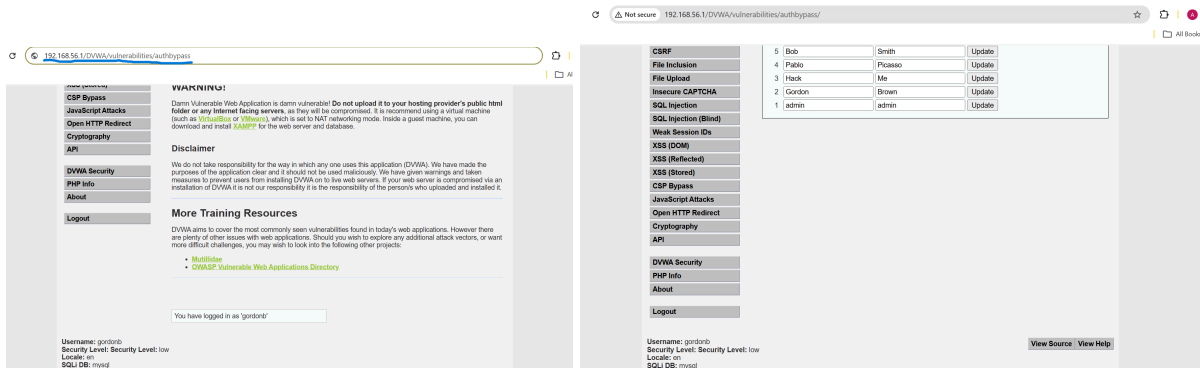
Successful authentication bypass was demonstrated during this assessment.

Evidence

— Authentication Request on low Severity

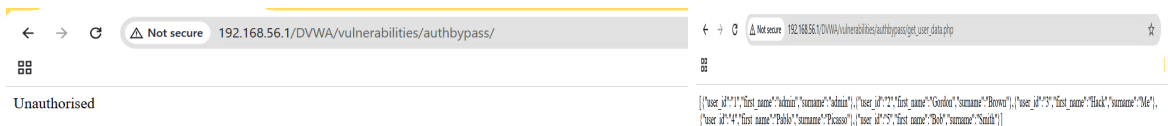


1stly login as another Web user (gordonb / abc123) ON LOW Severity



After that try to bypass access by changing URL & get access to user data even as **gordonb**

— Authentication Request on Medium Severity



1stly Request denied than Try exact user data file path & get access

❖ Session Management Testing

Objective

Session management was reviewed to identify potentially insecure handling of authenticated sessions.

Testing Performed

Burp Suite HTTP history was used to inspect authenticated DVWA requests and their associated cookies.

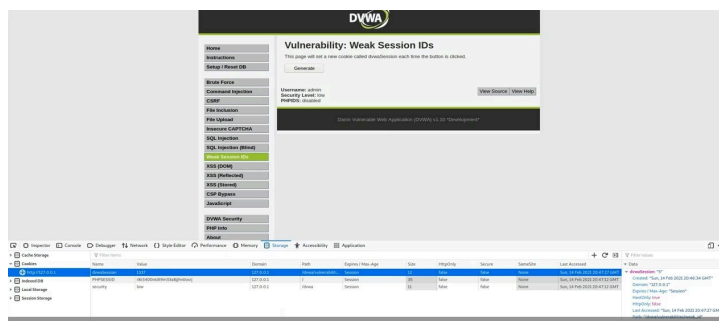
The application's session identifier was observed through the **PHPSESSID** cookie.

Observation

The application uses a PHP session cookie to maintain authenticated state.

Session-related attributes and behavior were inspected during testing.

Result



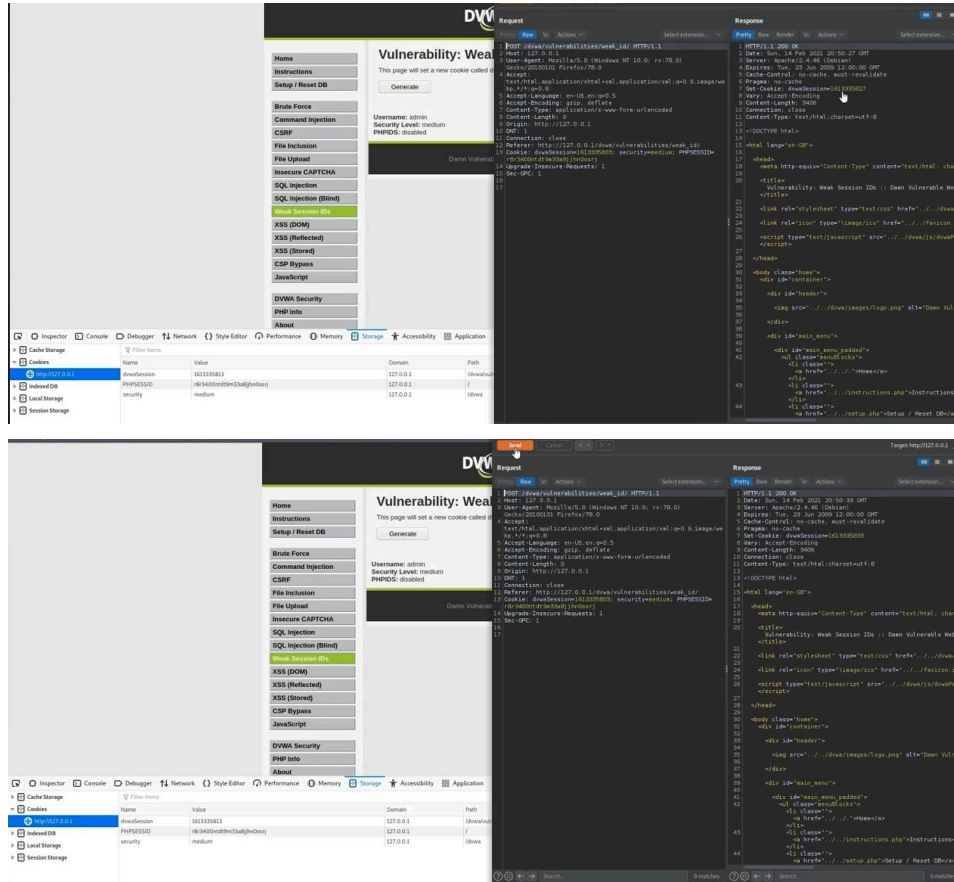
Recommendations

A production application should:

- Regenerate session identifiers after authentication.
- Set the **HttpOnly** cookie attribute.
- Set the **Secure** attribute when HTTPS is used.
- Configure an appropriate **SameSite** policy.
- Expire inactive sessions.
- Provide server-side session invalidation on logout.
- Avoid exposing session identifiers unnecessarily.
- Use HTTPS for all authenticated communication.

Evidence:

Figure 11 — Session Cookie Observed in Burp Suite



On sending every request Session Id regenerating Observed

❖ File Upload Testing

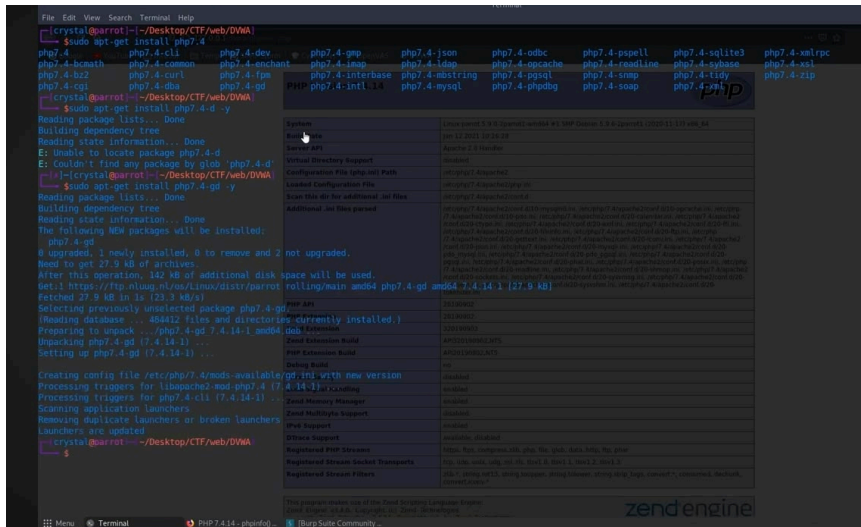
Objective

The file-upload functionality was considered as part of the assessment scope to determine whether the application accepts unsafe file types.

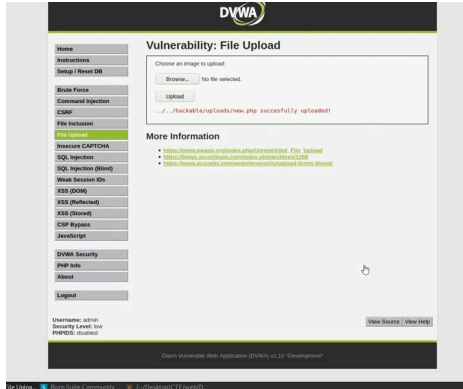
Testing



At First error “GD is not Installed” appear



After Installed gd



If a vulnerability was successfully demonstrated, document:

- Accepted file type
- Request/response
- Upload location
- Whether server-side execution was possible
- Security impact

If no vulnerability was demonstrated, state:

"No confirmed file-upload vulnerability was established during the available testing evidence."

14. Findings Summary

ID	Finding	Severity	Status
F-01	SQL Injection	low/High	Confirmed
F-02	Reflected XSS	low/Medium	Confirmed
F-03	Stored XSS	low/High	Confirmed
F-04	Cross-Site Request Forgery	low/Medium	Confirmed
F-05	Authentication Bypass	low/Medium	Confirmed
F-06	Session Management	low/Medium	Confirmed
F-07	File Upload	low/Medium	Confirmed

❖ Disclaimer

This assessment was performed solely against an intentionally vulnerable DVWA installation within an authorized local testing environment.

End of Report

Prepared by: Aliza Javed

Project: CloudExify Cybersecurity Project 3

Assessment: Advanced Web Application Penetration Testing

Target: DVWA

Environment: Authorized Local Lab